

Filière : Systèmes d'Information et Big Data (ISIBD)

RAPPORT DE PROJET LOGICIEL

Application de Gestion de Catalogue Produits avec Persistance Hibernate
ORM et Interface Graphique JavaFX

Réalisé par :
EL AZRAK Mohammed

Encadré par :
HRIMECH

Table des Matières

1	Introduction Générale et Contexte	2
1.1	Objectifs Spécifiques	2
2	Architecture et Conception du Système	3
2.1	Architecture en N-Couches (MVC Étendu)	3
2.2	Modélisation de l'Entité et Persistance	3
3	Implémentation Technique et Fonctionnalités	4
3.1	Gestion des Transactions dans la Couche Service	4
3.2	Contrôleur et Filtrage Dynamique en Temps Réel	4
3.3	Sécurisation de la Suppression	4
3.4	Modernisation Visuelle via CSS	4
4	Annexes : Scripts et Configurations	5
4.1	Script SQL de Création Logique	5
4.2	Fichier de Configuration Hibernate	5
5	Conclusion et Perspectives Évolutives	6
5.1	Perspectives d'Évolution (Big Data & Data Engineering)	6

1. Introduction Générale et Contexte

Dans le cadre de la modernisation des outils de gestion des Petites et Moyennes Entreprises (PME), la manipulation efficace des catalogues de produits constitue un enjeu stratégique majeur. Les architectures logicielles modernes exigent une séparation stricte entre la logique de présentation visuelle, les règles métiers, et les mécanismes de persistance des données.

Ce projet a pour objectif la conception et le développement d'une application desktop robuste de gestion de catalogue produits pour la structure "PME Distribution". L'application doit permettre d'effectuer l'ensemble des opérations CRUD (Create, Read, Update, Delete) sur les produits de manière fluide et sécurisée.

Afin de répondre à ces exigences de manière professionnelle, la solution technique retenue s'appuie sur le langage **Java 17+**, le framework de persistance objet-relationnel **Hibernate ORM** connecté à une base de données **MySQL**, et la bibliothèque graphique **JavaFX** pour la conception d'une interface utilisateur réactive et ergonomique.

1.1. Objectifs Spécifiques

- **Persistance Automatisée** : Éliminer l'écriture manuelle de requêtes SQL complexes en tirant parti du mapping objet-relationnel (ORM) d'Hibernate.
- **IHM Réactive** : Concevoir une interface basée sur le standard FXML permettant une synchronisation en temps réel entre le formulaire de saisie et la table d'affichage.
- **Recherche Prédictive** : Implémenter un système de filtrage dynamique en cours de saisie pour optimiser l'expérience utilisateur (UX).
- **Sécurisation des Opérations** : Intégrer des verrous de validation et des boîtes de dialogue de confirmation pour prévenir toute suppression accidentelle de données.

2. Architecture et Conception du Système

L'application adopte une architecture découpée en couches distinctes afin de garantir la maintenabilité, la réutilisabilité du code et une séparation stricte des responsabilités (Clean Code).

2.1. Architecture en N-Couches (MVC Étendu)

Le système est structuré autour des composantes suivantes :

1. **La Couche Modèle (Model)** : Représentée par l'entité POJO `Produit.java`. Elle définit la structure de la donnée et intègre les annotations JPA de mapping.
2. **La Couche Accès aux Données et Métier (Service)** : Matérialisée par `ProduitService.java`. Elle fait office de chef d'orchestre, gérant l'ouverture des sessions Hibernate, les transactions SQL, ainsi que les rollbacks en cas d'anomalie.
3. **La Couche Contrôleur (Controller)** : Représentée par `ProduitController.java`. Elle capte les événements de l'interface utilisateur, manipule les listes observables et délègue les traitements à la couche Service.
4. **La Couche Présentation (Vue)** : Composée du fichier descriptif `produit.fxml` et enrichie graphiquement par la feuille de style `style.css`.

2.2. Modélisation de l'Entité et Persistance

L'entité `Produit` est mappée directement sur une table relationnelle MySQL. L'identifiant technique de type `Long` est configuré en auto-incrémentation native. Le schéma ci-dessous résume les propriétés de l'objet métier :

Tableau 1: Structure de l'entité de données `Produit`

Attribut Java	Type Java	Colonne SQL	Contrainte
id	Long	id	Primary Key, Auto-increment
code	String	code	Unique, Not Null
libelle	String	libelle	Not Null
type	String	type	Not Null
quantiteStock	int	quantite_stock	Default 0
disponibilite	boolean	disponibilite	Not Null

La configuration de la connexion à la base de données est centralisée dans le fichier `hibernate.cfg.xml`. Grâce à la propriété `hibernate.hbm2ddl.auto` configurée sur la valeur `update`, Hibernate analyse le schéma existant dans MySQL au démarrage et crée ou met à jour la table de manière totalement transparente sans intervention manuelle du développeur.

3. Implémentation Technique et Fonctionnalités

Cette section détaille les choix d'implémentation pour les fonctionnalités majeures de l'application, notamment la couche de service et la logique du contrôleur.

3.1. Gestion des Transactions dans la Couche Service

La classe `ProduitService` encapsule l'ensemble des opérations CRUD. Chaque interaction d'écriture (ajout, modification, suppression) est sécurisée à l'intérieur d'un bloc transactionnel contrôlé. En cas d'erreur ou d'exception levée, la transaction effectuée immédiatement un `rollback()` afin d'éviter toute corruption ou état incohérent au sein de la base de données MySQL.

L'extraction de la liste complète des produits s'effectue via une requête HQL (Hibernate Query Language) propre, abstraite de toute syntaxe SQL spécifique au SGBD :

```
session.createQuery("from Produit", Produit.class).list();
```

3.2. Contrôleur et Filtrage Dynamique en Temps Réel

Le `ProduitController` orchestre l'IHM JavaFX. Une attention particulière a été portée sur deux fonctionnalités ergonomiques avancées :

- **Écouteur de Sélection (Table → Formulaire) :** Un listener est greffé sur la propriété de sélection du `TableView`. Dès que l'utilisateur clique sur une ligne du tableau, le formulaire de gauche se pré-remplit instantanément avec les données du produit sélectionné, mémorisant l'instance dans une variable de contexte.
- **Recherche Prédictive Synchronne :** Pour éviter des requêtes répétitives et lourdes vers le serveur MySQL, la barre de recherche utilise une `FilteredList` enveloppant la liste d'origine (`ObservableList`). Un listener écoute chaque caractère tapé par l'utilisateur et applique un prédicat de filtrage combiné (recherche par libellé ou par type) directement en mémoire.

3.3. Sécurisation de la Suppression

Afin de prémunir l'application contre les fausses manipulations, la fonction de suppression intègre une validation bloquante via une boîte de dialogue modale d'alerte (`Alert.AlertType.CONFIRMATION`). Le processus métier ne valide la transaction Hibernate que si et seulement si l'utilisateur clique explicitement sur le bouton OK.

3.4. Modernisation Visuelle via CSS

Le style d'interface par défaut de JavaFX (Modena) a été surchargé par une feuille de style `style.css` personnalisée. Ce fichier applique des codes visuels contemporains : coins arrondis pour les champs de saisie (`-fx-background-radius: 5px`), couleurs de boutons contrastées et survol dynamique (`:hover`), ainsi qu'une coloration de sélection de ligne adoucie pour améliorer le confort visuel de l'opérateur.

4. Annexes : Scripts et Configurations

4.1. Script SQL de Création Logique

Bien qu'Hibernate génère le schéma de manière autonome, voici le script SQL équivalent standard permettant de déployer la structure relationnelle sur n'importe quel SGBD MySQL externe :

Listing 1: Script de structure de base de données (script.sql)

```
1  -- Creation de la base de donnees
2  CREATE DATABASE IF NOT EXISTS gestion_produits
3  CHARACTER SET utf8mb4
4  COLLATE utf8mb4_general_ci;
5
6  USE gestion_produits;
7
8  -- Creation de la table produit
9  CREATE TABLE IF NOT EXISTS produit (
10     id BIGINT AUTO_INCREMENT PRIMARY KEY,
11     code VARCHAR(50) NOT NULL UNIQUE,
12     libelle VARCHAR(150) NOT NULL,
13     type VARCHAR(50) NOT NULL,
14     quantite_stock INT NOT NULL DEFAULT 0,
15     disponibilite BOOLEAN NOT NULL DEFAULT TRUE
16 ) ENGINE=InnoDB;
```

4.2. Fichier de Configuration Hibernate

Le fichier de configuration XML établit le pont entre l'application Java et le moteur MySQL local en injectant les identifiants de connexion requis.

Listing 2: Fichier de configuration central hibernate.cfg.xml

```
1  <!DOCTYPE hibernate-configuration PUBLIC
2      "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
3      "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
4  <hibernate-configuration>
5      <session-factory>
6          <property name="hibernate.connection.driver_class">com.mysql.cj.
7              jdbc.Driver</property>
8          <property name="hibernate.connection.url">jdbc:mysql://
9              localhost:3306/gestion_produits?serverTimezone=UTC</property>
10         <property name="hibernate.connection.username">root</property>
11         <property name="hibernate.connection.password">
12             votre_mot_de_passe</property>
13
14         <property name="hibernate.dialect">org.hibernate.dialect.
15             MySQLDialect</property>
16         <property name="hibernate.show_sql">>true</property>
17         <property name="hibernate.hbm2ddl.auto">update</property>
18
19         <mapping class="com.mycompany.hibernate_project.model.Produit"/>
20     </session-factory>
21 </hibernate-configuration>
```

5. Conclusion et Perspectives Évolutives

Le projet d'application de gestion de catalogue produits a permis de mettre en œuvre de bout en bout un flux complet d'ingénierie logicielle Java. L'intégration combinée d'Hibernate ORM et de JavaFX démontre une grande efficacité : le premier abstrait totalement la complexité relationnelle et la gestion des connexions JDBC brutes, tandis que le second offre un framework graphique performant pour concevoir des applications de bureau interactives et fluides.

Toutes les exigences fonctionnelles du cahier des charges ont été validées avec succès : les données se synchronisent de manière persistante dans MySQL, le filtrage en temps réel est opérationnel, et l'interface utilisateur offre un rendu moderne et sécurisé face aux actions critiques de suppression.

5.1. Perspectives d'Évolution (Big Data & Data Engineering)

En tant qu'étudiant de la filière **Systèmes d'Information et Big Data (ISIBD)**, plusieurs verrous technologiques complémentaires pourraient être levés pour transformer cette application de gestion en un véritable outil d'aide à la décision :

- **Module Analytics / Dashboard** : Intégrer des composants graphiques natifs JavaFX (BarChart, PieChart) pour afficher des indicateurs clés de performance (KPI) en temps réel, tels que la valorisation financière totale du stock ou la répartition sectorielle des produits par catégorie.
- **Pipeline d'Extraction de Données** : Développer un module d'exportation automatique des données du catalogue aux formats standardisés CSV ou JSON. Cela permettrait de créer une passerelle directe vers un environnement d'analyse de données Python (exploitant les bibliothèques Pandas, Seaborn et Scikit-Learn) pour réaliser des prévisions de rupture de stock ou des analyses prédictives sur les tendances de ventes.
- **Architecture Multi-utilisateurs** : Faire évoluer le modèle vers une architecture distribuée, intégrant un système de gestion des droits et rôles des utilisateurs (Administrateur, Gestionnaire de Stock, Vendeur) pour sécuriser l'accès aux données de l'entreprise.